Empirical Analysis of Insertion Sort

Algorithms: Analysis and Design - Mini Project

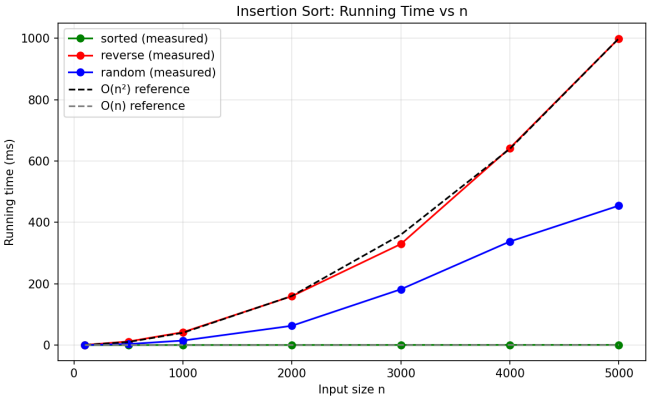
1. Setup

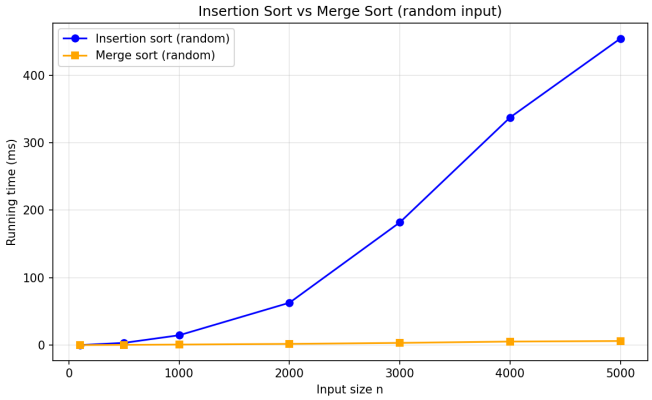
Insertion sort was implemented in Python with counters for comparisons and shifts. It was timed for n = 100 to 5,000 on three input types: sorted, reverse-sorted and random. Each time is the average of 3 runs. Merge sort was timed on the same inputs for comparison.

2. Results (n = 5,000)

Input	Insertion time	Comparisons	Shifts	Merge sort time
Sorted	0.52 ms	4,999	0	5.70 ms
Random	454.45 ms	6,126,576	6,121,585	6.06 ms
Reverse	999.52 ms	12,497,500	12,497,500	3.35 ms

3. Analysis





Sorted input (best case, O(n)): the inner loop stops after one comparison per element, so there are n-1 comparisons and 0 shifts. The measured curve is a straight line, matching the O(n) reference.
Reverse input (worst case, $O(n^2)$): every element moves all the way to the front, giving n(n-1)/2 comparisons and shifts (12,497,500 at n = 5,000). Doubling n from 2,000 to 4,000 raised the time from 159 ms to 642 ms (about 4x), which is quadratic growth.
Random input (average case, $O(n^2)$): about half the shifts of the worst case, and the time is roughly half too (454 ms vs 999 ms).
Shifts drive the cost: comparisons and shifts are nearly equal on random and reverse inputs, so total work is dominated by moving elements.

4. Comparison with merge sort

Merge sort is O(n log n) in every case. At n = 5,000 on random input it took 6 ms versus 454 ms for insertion sort (about 75x faster), and on reverse input 3 ms versus 1,000 ms. The gap grows as n grows. On already sorted input, however, insertion sort was about 10x faster (0.5 ms vs 5.7 ms) because it runs in O(n) and has no recursion or copying overhead.

5. Conclusion

Insertion sort needs no extra memory (O(1) space) and is excellent for small or nearly sorted inputs, but its quadratic cost makes it impractical for large random data. Merge sort scales far better at the price of O(n) extra memory, which is the time-space trade-off.